

The tutor's four numbers

What they do, and what happens when they are wrong

28 September 2026

1. What this part is about

The tutor in our simulator does not watch the student learn. It only sees answers, right or wrong, and from those it keeps a running guess about whether each concept is known. That guess is built from four numbers.

This document is about those four numbers. It asks three things in order. What does each number actually do? What goes wrong when the tutor's numbers do not match the student's? And if students differ from each other, what single set of numbers should the tutor use for everybody?

Everything here runs on a small practice curriculum of eight concepts arranged in four layers, and every figure comes from a file of results that one command regenerates.

2. The four numbers, in plain words

	Meaning	Default
$p(L_0)$	probability the concept is known before any question	0.15
$p(T)$	probability an unknown concept becomes known at an attempt	0.20
$p(G)$	probability of a correct answer while it is unknown	0.25
$p(S)$	probability of a wrong answer while it is known	0.10

Read them as sentences about a student:

- $p(L_0)$: some students already know a thing before you teach it.
- $p(T)$: each time you ask about something they do not know, there is a chance they pick it up.
- $p(G)$: someone who does not know it can still answer correctly by luck.
- $p(S)$: someone who does know it can still slip and get it wrong.

The last two are why this is hard. A right answer is not proof of knowing, and a wrong answer is not proof of not knowing. The tutor has to weigh evidence rather than read it off.

Two things matter to a teacher, so those are what we measure. How long does the tutor take to notice that a concept has been learned, counted in questions asked about that concept. And how often does it announce mastery too early, before the student has actually learned it.

3. What each number does

Here the student and the tutor use the *same* values, so the tutor is not wrong about anything. We just change the numbers and watch.

	swept over	questions to notice	declared too early
$p(L_0)$	0.05 to 0.50	3.1 to 2.4	0.050 to 0.075
$p(T)$	0.05 to 0.40	4.3 to 1.8	0.076 to 0.042
$p(G)$	0.05 to 0.45	2.1 to 3.8	0.000 to 0.058
$p(S)$	0.02 to 0.30	2.9 to 3.7	0.025 to 0.071

Three of these behave the way common sense says they should.

When learning is faster, $p(T)$ large, the tutor notices sooner and announces too early less often. Everything gets easier at once.

When guessing is easier, $p(G)$ large, lucky right answers look like knowledge, so the tutor is fooled more often and takes longer to be sure.

When slipping is more likely, $p(S)$ large, a student who does know something keeps answering wrongly, so the tutor holds back and takes longer.

The fourth, $p(L_0)$, is the odd one. Raising it makes the tutor notice faster but also announce too early more often. That is not a contradiction. If many students already know things, starting optimistic is usually right and occasionally badly wrong.

There is one catch worth stating. The tutor announces mastery when its guess passes a bar, and passing that bar takes a whole number of correct answers in a row. So these curves are steps, not smooth slopes. A change in a parameter that does not move that whole number changes nothing at all, and a change that does move it shifts everything at once. That is why some of these lines bend the wrong way in places.

4. When the tutor's numbers are wrong: one student

Now we keep the student fixed and give the tutor the wrong value on purpose, by a known amount, one number at a time. If the tutor's guess is honest, its error should be zero when it is right and grow as it is made more wrong.

tutor wrong about	gap when right	worst value	gap there	declared too early
$p(T)$	+0.002	0.05 (-0.15)	+0.133	0.013
$p(G)$	+0.002	0.05 (-0.20)	-0.063	0.171
$p(S)$	+0.002	0.30 (+0.20)	-0.037	0.070

That is what happens. The middle column is near zero for all three, which is the check that the machinery is sound.

Beyond that, the three numbers do not matter equally. Being wrong about $p(T)$ moves the tutor's belief about twice as much as being wrong about either of the others. If you only have time to measure one thing about a new group of students, measure how fast they learn.

The more useful finding is about direction. Being wrong about $p(G)$ and being wrong about $p(T)$ push announcing-too-early in *opposite* ways. A tutor that thinks students guess more than they do becomes cautious and announces less. A tutor that thinks students learn faster than they do becomes hasty and announces more. So two different mistakes can cancel, and a tutor can look well behaved while being wrong about both.

We deliberately do not rank $p(T)$ against $p(G)$ here. We tried, and the ranking flipped depending on which side of zero we looked at and which random seed we used. It is not a stable result and we are not reporting one.

5. When every student is different

Real students are not identical. Here each simulated student draws their own values, and the tutor uses one set for everybody, sitting at the middle of the group.

how different the students are	belief gap	declared too early	questions to notice
± 0.00	+0.002	0.046	3.06
± 0.05	+0.001	0.041	3.07
± 0.10	-0.008	0.058	3.11
± 0.15	-0.017	0.075	3.13

The first row is the control: everybody matches the tutor. Reading down, the tutor gets steadily worse, but slowly. Even when students vary quite a lot, the tutor’s belief is off by under two points and it announces too early on under eight percent of concepts. One model for a mixed group is workable, not ideal.

6. So what should the tutor assume?

The obvious follow-up: if we cannot give every student their own numbers, which single set is best, and how much are we losing by not adapting?

The second half needs something to compare against. So we also run an *oracle* tutor that is simply told each student’s true values. No real tutor can do this. It is there to mark the ceiling: whatever the oracle gains is the most that adapting could ever be worth.

vary	spread	best fixed	its Brier	oracle	buys	95% CI	resolved
p-S	± 0.05	0.10	0.0835	0.0834	+0.0002	± 0.0005	no
p-S	± 0.10	0.10	0.0836	0.0834	+0.0002	± 0.0013	no
p-S	± 0.15	0.10	0.0854	0.0835	+0.0019	± 0.0013	yes
p-G	± 0.05	0.25	0.0826	0.0832	-0.0006	± 0.0016	no
p-G	± 0.10	0.25	0.0824	0.0835	-0.0011	± 0.0018	no
p-G	± 0.15	0.25	0.0829	0.0828	+0.0001	± 0.0020	no

Two answers.

The best fixed tutor is always the one sitting at the middle of the group. We could not find a value that beat it, at any spread, for either number. So there is no clever offset to discover.

And adapting is worth very little. Knowing each student’s own $p(S)$ helps only when students vary a lot, and then only by about two percent. Knowing each student’s own $p(G)$ never helped measurably at all.

One honest caveat. “No measurable difference” is not the same as “no difference”. We did not set a threshold in advance for how small a gain would count as none. All we can say is that if adapting helps, it helps by less than this study could see.

We should also say why we score this on the Brier score rather than something simpler. We first used the average gap between what the tutor believes and what is true, and that was a mistake. A tutor that is too confident about half its students and not confident enough about the other half scores perfectly on that measure, because the two halves cancel. That is exactly the error adapting would fix, so the measure was blind to the question being asked. The Brier score does not cancel, and it correctly picks out the true value when we feed it one.

7. What we are not claiming

All of this is on one small curriculum of eight concepts. Nothing has been rerun on the larger one, so none of it is established beyond this graph.

Every number describes a simulated student, not a person. No result here has been checked against a real learner.

The bar for announcing mastery is fixed at 0.9 and is not swept, and since it is a whole-number-of-correct-answers effect, some of these comparisons would move if the bar moved.

8. Running it again

`python3 make_sanity.py` reruns every study and writes the result files. `python3 make_tables.py` turns those files into every table in this document, so no number here is typed by hand. The command `python3 make_tables.py --check` fails if the document and the results ever drift apart. Seeds are fixed, so a rerun gives the same numbers.